# Linux Kernel Programming

Linux Kernel programming

# Agenda

- LED Driver New way
- Platform Driver
- Platform Driver on RPI
- Misc Driver

# Character Driver – New Way

- Character devices are represented in the kernel as instances of struct cdev, defined in include/linux/cdev.h:

```
struct cdev {
    struct kobject kobj;
    struct module *owner;
    const struct file_operations *ops;
    struct list_head list;
    dev_t dev;
    unsigned int count;
};
```

# Character Driver – New Way

- The kernel holds the numbers that identify a device in dev_t type variables, which are simply u32 (32-bit unsigned long). The major is represented with only 12 bits, whereas the minor is coded on the 20 remaining bits.

- in include/linux/kdev_t.h, given a dev_t type variable, you may needto extract the minor or the major. The kernel provides a macro for these purposes:

  **MAJOR(dev_t dev);**

  **MINOR(dev_t dev);**

- On the other hand, you may have a minor and a major, and need to build a dev_t. The macro you should use is **MKDEV(int major, int minor):**

```
#define MINORBITS    20
#define MINORMASK    ((1U << MINORBITS) - 1)
#define MAJOR(dev)   ((unsigned int) ((dev) >> MINORBITS))
#define MINOR(dev)   ((unsigned int) ((dev) & MINORMASK))
#define MKDEV(ma,mi) (((ma) << MINORBITS) | (mi))
```

# Character Driver – New Way

- The device is registered with a major number that identifies the device, and a minor, which you may use as an array index to a local list of devices, since one instance of the same driver may handle several devices while different drivers may handle different devices of the same type.

# Character Driver – New Way

- Device numbers identify device files across the system. That means there are two ways to allocate these device numbers (actually major and minor):

- **Statically:** Guessing a major not yet used by another driver using the register_chrdev_region() function. You should avoid using this as much as possible. Its prototype looks this:

```
int register_chrdev_region(dev_t first, unsigned int count, \
                           char *name);
```

This method returns 0 on success, or a negative error code on failure. first is made of the major number that we need along with the first minor of the desired range. You should use MKDEV(ma,mi). count is the number of consecutive device numbers required, and name should be the name of the associated device or driver.

- **Dynamically:** Letting the kernel do the job for us, using the alloc_chrdev_region() function. This is the recommended way to obtain a valid device number. Its prototype is as follows:

```
int alloc_chrdev_region(dev_t *dev, unsigned int firstminor, \
                        unsigned int count, char *name);
```

# Character Driver – New Way

This method returns 0 on success, or a negative error code on failure. dev is the only output parameter. It represents the first number the kernel assigned. firstminor is the first of the requested range of minor numbers, count the number of minors you require, and name should be the name of the associated device or driver.

- The difference between the two is that with the former, you should know in advance what number we need. This is registration: you tell the kernel what device numbers you want. This may be used for pedagogic purposes, and works as long as the only user of the driver is you. When it comes to loading the driver on another machine, there is no guarantee the chosen number is free on that machine, and this will lead to conflicts and trouble. The second method is cleaner and much safer, since the kernel is responsible for guessing the right numbers for us.

# Character Driver – New Way

- Removing of allocated chardev region is done using function

  **void unregister_chrdev_region(dev_t first, unsigned int count);**

# Character Driver – New Way

- To reserve my_minor_count devices, starting with my_major major and my_first_minor minor. The first parameter of the register_chrdev_region() function is the first identifier of the device. The successive identifiers can be retrieved using the MKDEV macro.

```
register_chrdev_region(MKDEV(my_major, my_first_minor), my_minor_count,
                       "my_device_driver");
```

- After assigning the identifiers, the character device will have to be initialized using the cdev_init() function and registered to the kernel using the cdev_add() function. The cdev_init() and cdev_add() functions will be called as many times as assigned device identifiers.

# Character Driver – New Way

- The following sequence registers and initializes MY_MAX_MINORS devices:

```
#include <linux/fs.h>
#include <linux/cdev.h>
#define MY_MAJOR       42
#define MY_MAX_MINORS  5
struct my_device_data {
    struct cdev cdev;
    /* my data starts here */
    [...]
};
struct my_device_data devs[MY_MAX_MINORS];
```

# Character Driver – New Way

```
const struct file_operations my_fops = {
    .owner = THIS_MODULE,
    .open = my_open,
    .read = my_read,
    .write = my_write,
    .release = my_release,
    .unlocked_ioctl = my_ioctl
};
```

# Character Driver – New Way

```
int init_module(void)
{
   int i, err;
   register_chrdev_region(MKDEV(MY_MAJOR, 0), MY_MAX_MINORS,
                          "my_device_driver");

for(i = 0; i < MY_MAX_MINORS; i++) {
        /* initialize devs[i] fields and register character devices */
        cdev_init(&devs[i].cdev, &my_fops);
        cdev_add(&devs[i].cdev, MKDEV(MY_MAJOR, i), 1);
    }
    return 0;
}
```

# Character Driver – New Way

- The following sequence deletes and unregisters the character devices:

```
void cleanup_module(void)
{
    int i;
    for(i = 0; i < MY_MAX_MINORS; i++) {
        /* release devs[i] fields */
        cdev_del(&devs[i].cdev);
    }
    unregister_chrdev_region(MKDEV(MY_MAJOR, 0), MY_MAX_MINORS);
}
```

# Character Driver – Device File

- The device file allows transparent communication between user-space applications and hardware.

- They are not normal “files”, but look like files from the program’s point of view: you can read from them, write to them, mmap() onto them, and so forth. When you access such a device “file,” the kernel recognizes the I/O request and passes it to a device driver, which performs some operation, such as reading data from a serial port or sending data to hardware.

- Device files (although inappropriately named, we will continue to use this term) provide a convenient way to access system resources without requiring the application programmer to know how the underlying device works. Under Linux, as with most Unix systems, device drivers themselves are part of the kernel.

- All device files are stored in /dev directory. Use ls command to browse the directory:

  **ls -l /dev/**

# Character Driver – Device File

- Each device on the system should have a corresponding entry in /dev. For example, /dev/ttyS0 corresponds to the first serial port, known as COM1 under MS-DOS; /dev/hda2 corresponds to the second partition on the first IDE drive. In fact, there should be entries in /dev for devices you do not have. The device files are generally created during system installation and include every possible device driver. They don't necessarily correspond to the actual hardware on your system. There are a number of pseudo-devices in /dev that don't correspond to any actual peripheral. For example, /dev/null acts as a byte sink; any write request to /dev/null will succeed, but the data are written will be ignored.

- When using ls -l to list device files in /dev, you'll see something like the following:

  crw--w---- 1 root tty 4, 0 Aug 15 10:40 tty0

  brw-rw---- 1 root disk 1, 0 Aug 15 10:40 ram0

First of all, note that the first letter of the permissions field is denoted that driver type. Device files are denoted either by b, for block devices, or c, for character devices.

# Character Driver – Device File

- Also, note that the size field in the ls -l listing is replaced by two numbers, separated by a comma. The first value is the major device number and the second is the minor device number. This we have discussed in the previous tutorial.

# Character Driver – Device File (Creation)

- We can create a dive file in two ways.
  - Manually
  - Automatically

# Character Driver – Device File (Manual Creation)

- We can create the device file manually by using mknod.

**mknod -m <permissions> <name> <device type> <major> <minor>**

**Where,**

**<name>** – your device file name that should have a full path (/dev/name)

**<device type>** – Put c or b

- c – Character Device
- b – Block Device

**<major>** – major number of your driver

**<minor>** – minor number of your driver

**-m <permissions>** – optional argument that sets the permission bits of the new device file to permissions

# Character Driver – Device File (Manual Creation)

**Example:**

**sudo mknod -m 666 /dev/etx_device c 246 0**

- If you don’t want to give permission, You can also use chmod to set the permissions for a device file after creation.

**Advantages**

- Anyone can create the device file using this method.
- You can create the device file even before loading the driver.

# Character Driver – Device File (Automatic Creation)

- The automatic creation of device files can be handled that creates/removes device nodes in the /dev directory dynamically. Just follow the below steps.
  - Include the header file linux/device.h and linux/kdev_t.h
  - Create the struct Class
  - Create Device with the class which is created by the above step

# Character Driver – Device File (Automatic Creation)

**Create the class**

- This will create the struct class for our device driver. It will create a structure under/sys/class/.

  **struct class * class_create (struct module *owner, const char *name);**

**Where,**

**owner** – pointer to the module that is to “own” this struct class

**name** – pointer to a string for the name of this class

- This is used to create a struct class pointer that can then be used in calls to device_create.

# Character Driver – Device File (Automatic Creation)

- Note, the pointer created here is to be destroyed when finished by making a call to class_destroy.

  **void class_destroy (struct class * cls);**

# Character Driver – Device File (Automatic Creation)

## Create Device

- This function can be used by char device classes. A struct device will be created in sysfs, registered to the specified class.

  **struct device *device_create (struct *class, struct device *parent, dev_t dev, void * drvdata, const char *fmt, ...);**

**Where,**

**class** – pointer to the struct class that this device should be registered to

**parent** – pointer to the parent struct device of this new device, if any

**devt** – the dev_t for the char device to be added

**drvdata** – the data to be added to the device for callbacks

**fmt** – string for the device's name

... – variable arguments

# Character Driver – Device File (Automatic Creation)

- A “dev” file will be created, showing the dev_t for the device, if the dev_t is not 0,0. If a pointer to a parent struct device is passed in, the newly created struct device will be a child of that device in sysfs. The pointer to the struct device will be returned from the call. Any further sysfs files that might be required can be created using this pointer.

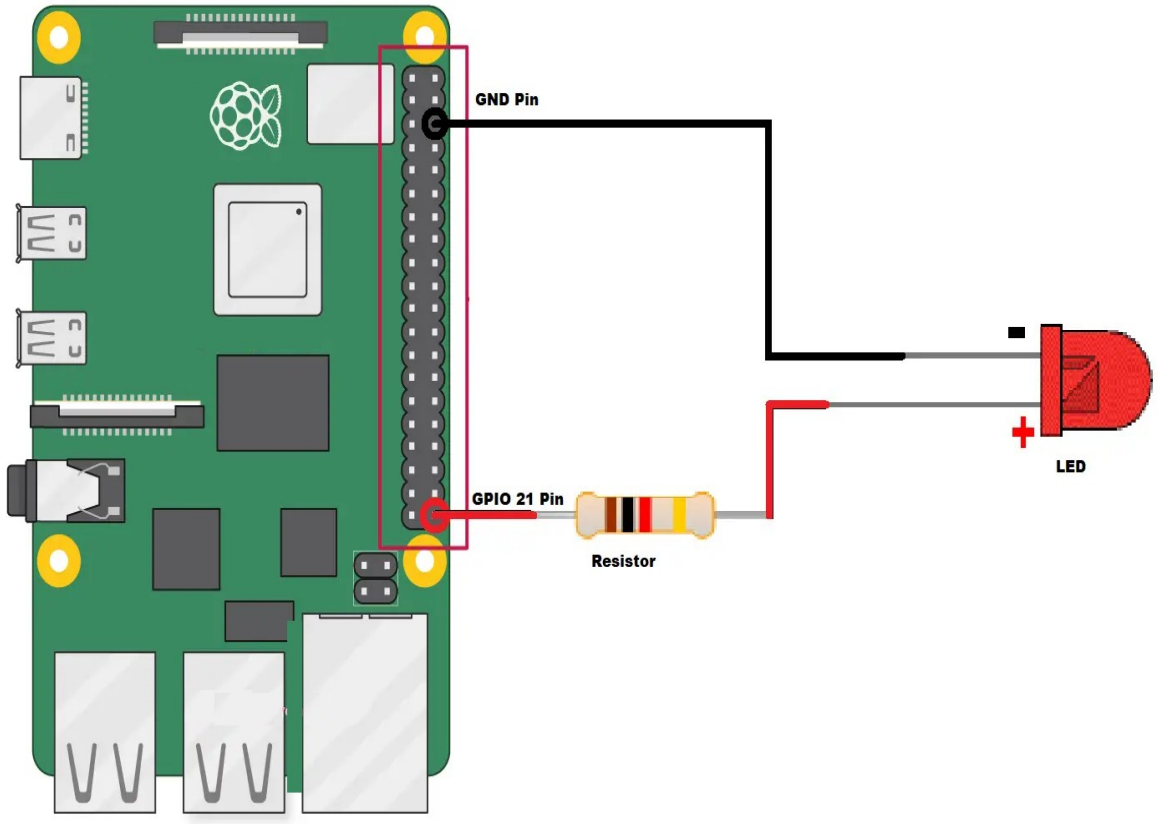
# Character Driver – Device File (Automatic Creation)

- Note, you can destroy the device using device_destroy().

  **void device_destroy (struct class * class, dev_t devt);**

If you don’t understand please refer to the below program, Then you will get some idea.

# Character Driver – Old Way – LED Driver IOCTL



# Platform Driver

- On embedded systems, devices are often not connected through a bus, allowing enumeration or hotplugging for these devices.

- However, you still want all of these devices to be part of the device model. Such devices, instead of being dynamically detected, must be statically described:
  - By direct instantiation of struct platform_device structures, as done on a few old ARM non-Device Tree based platforms. Definition is done in the board-specific or SoC specific code.
  - In the Device Tree, a hardware description file used on some architectures. The device drivers match with the physical devices described in the .dts file. After this matching the driver´s probe() function is called. An .of_match_table has to be included in the driver´s code to allow this matching.

# Platform Driver

- Amongst the non-discoverable devices, a huge family is directly part of a system-on-chip: UART controllers, Ethernet controllers, SPI controllers, graphic or audio devices, etc. In the Linux kernel, a special bus, called the platform bus has been created to handle such devices.

- It supports platform drivers that handle platform devices. It works like any other bus (USB, PCI), except that devices are enumerated statically instead of being discovered dynamically.

- Each platform driver is responsible for instantiating and registering an instance of a struct platform_driver structure within the device model core.

- Platform drivers follow the standard driver model convention, where discovery/enumeration is handled outside the drivers, and drivers provide probe() and remove() methods. They support power management and shutdown notifications using the standard conventions.

# Platform Driver

- The most important members of the struct platform_driver are shown below:

```
struct platform_driver {
    int (*probe)(struct platform_device *);
    int (*remove)(struct platform_device *);
    void (*shutdown)(struct platform_device *);
    int (*suspend)(struct platform_device *, pm_message_t state);
    int (*suspend_late)(struct platform_device *, pm_message_t state);
    int (*resume_early)(struct platform_device *);
    int (*resume)(struct platform_device *);
    struct device_driver driver;
};
```

# Platform Driver

- In the struct platform_driver you can see a function pointer variable that points to a function named probe(). The probe() function is called when the "bus driver" pairs the "device" to the "device driver". The probe() function is responsible of initializing the device and registering it in the appropriate kernel framework:
  - The probe() function gets a pointer to a device structure as an argument (for example, struct pci_dev *, struct usb_dev *, struct platform_device *, struct i2c_client *).
  - It initializes the device, maps I/O memory, allocates buffers, registers interrupt handlers, timers, and so on…
  - It registers the device to specific framework(s), (for example, network, misc, serial, input, industrial).

- The suspend()/resume() functions are used by devices that support low power management features.

# Platform Driver

- The platform driver responsible for the platform device should be registered to the platform core using the platform_driver_register(struct platform_driver *drv) function. Register your platform driver in your module init() function, and unregister your platform driver in the module exit() function.

- We can also use the module_platform_driver(my_platform_driver) macro. This is a helper macro for drivers that don't do anything special in module init()/exit(). This eliminates a lot of boilerplate. Each module may only use this macro once, and calling it replaces module_init() and module_exit().

# Platform Driver

- On embedded systems, devices are often not connected through a bus allowing enumeration, hotplugging, and providing unique identifiers for devices.
- For example, the devices on I2C buses or SPI buses, or the devices directly part of the system-on-chip.
- However, we still want all of these devices to be part of the device model.
- Such devices, instead of being dynamically detected, must be statically described in either:
  - The kernel source code
  - The *Device Tree*, a hardware description file used on some architectures.
  - BIOS ACPI tables (x86/PC architecture)

# Platform Driver

- Amongst the non-discoverable devices, a huge family are the devices that are directly part of a system-on-chip: UART controllers, Ethernet controllers, SPI or I2C controllers, graphic or audio devices, etc.
- In the Linux kernel, a special bus, called the **platform bus** has been created to handle such devices.
- It supports **platform drivers** that handle **platform devices**.
- It works like any other bus (USB, PCI), except that devices are enumerated statically instead of being discovered dynamically.

# Platform Driver

The driver implements a `struct platform_driver` structure (example taken from `drivers/tty/serial/imx.c`, simplified)

```
static struct platform_driver serial_imx_driver = {
        .probe          = serial_imx_probe,
        .remove         = serial_imx_remove,
        .id_table       = imx_uart_devtype,
        .driver         = {
                .name   = "imx-uart",
                .of_match_table = imx_uart_dt_ids,
                .pm     = &imx_serial_port_pm_ops,
        },
};
```

# Platform Driver

... and registers its driver to the platform driver infrastructure

```
static int __init imx_serial_init(void) {
    ret = platform_driver_register(&serial_imx_driver);
}

static void __exit imx_serial_cleanup(void) {
    platform_driver_unregister(&serial_imx_driver);
}

module_init(imx_serial_init);
module_exit(imx_serial_cleanup);
```

Most drivers actually use the `module_platform_driver()` macro when they do nothing special in `init()` and `exit()` functions:

```
module_platform_driver(serial_imx_driver);
```

# Platform Driver

- As platform devices cannot be detected dynamically, they are defined statically
  - By direct instantiation of `struct platform_device` structures, as done on a few old ARM platforms. Definition done in the board-specific or SoC specific code.
  - By using a *device tree*, as done on Power PC (and on most ARM platforms) from which `struct platform_device` structures are created
- Example on ARM, where the instantiation was done in `arch/arm/mach-imx/mx1ads.c`

```
static struct platform_device imx_uart1_device = {
    .name = "imx-uart",
    .id = 0,
    .num_resources = ARRAY_SIZE(imx_uart1_resources),
    .resource = imx_uart1_resources,
    .dev = {
        .platform_data = &uart_pdata,
    }
};
```

# Platform Driver

- The device was part of a list

```
static struct platform_device *devices[] __initdata = {
    &cs89x0_device,
    &imx_uart1_device,
    &imx_uart2_device,
};
```

- And the list of devices was added to the system during board initialization

```
static void __init mx1ads_init(void)
{
    [...]
    platform_add_devices(devices, ARRAY_SIZE(devices));
}

MACHINE_START(MX1ADS, "Freescale MX1ADS")
    [...]
    .init_machine = mx1ads_init,
MACHINE_END
```

# Platform Driver

- Each device managed by a particular driver typically uses different hardware resources: addresses for the I/O registers, DMA channels, IRQ lines, etc.
- Such information can be represented using `struct resource`, and an array of `struct resource` is associated to a `struct platform_device`
- Allows a driver to be instantiated for multiple devices functioning similarly, but with different addresses, IRQs, etc.

# Platform Driver

```
static struct resource imx_uart1_resources[] = {
    [0] = {
        .start = 0x00206000,
        .end = 0x002060FF,
        .flags = IORESOURCE_MEM,
    },
    [1] = {
        .start = (UART1_MINT_RX),
        .end = (UART1_MINT_RX),
        .flags = IORESOURCE_IRQ,
    },
};
```

# Platform Driver

- When a `struct platform_device` was added to the system using `platform_add_devices()`, the `probe()` method of the platform driver was called
- This method is responsible for initializing the hardware, registering the device to the proper framework (in our case, the serial driver framework)
- The platform driver has access to the I/O resources:

```
res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
base = ioremap(res->start, PAGE_SIZE);
sport->rxirq = platform_get_irq(pdev, 0);
```

# Platform Driver

- In addition to the well-defined resources, many drivers require driver-specific information for each platform device
- Such information could be passed using the `platform_data` field of `struct device` (from which `struct platform_device` inherits)
- As it is a `void *` pointer, it could be used to pass any type of information.
  - Typically, each driver defines a structure to pass information through `struct platform_data`

# Platform Driver

- The i.MX serial port driver defines the following structure to be passed through `struct platform_data`

```
struct imxuart_platform_data {
    int (*init)(struct platform_device *pdev);
    void (*exit)(struct platform_device *pdev);
    unsigned int flags;
    void (*irda_enable)(int enable);
    unsigned int irda_inv_rx:1;
    unsigned int irda_inv_tx:1;
    unsigned short transceiver_delay;
};
```

- The MX1ADS board code instantiated such a structure

```
static struct imxuart_platform_data uart_pdata = {
    .flags = IMXUART_HAVE_RTSCTS,
};
```

# Platform Driver

- The uart_pdata structure was associated to the struct platform_device structure in the MX1ADS board file (the real code was slightly more complicated)

```
struct platform_device mx1ads_uart1 = {
    .name = "imx-uart",
    .dev {
         .platform_data = &uart_pdata,
    },
    .resource = imx_uart1_resources,
    [...]
};
```

- The driver can access the platform data:

```
static int serial_imx_probe(struct platform_device *pdev)
{
    struct imxuart_platform_data *pdata;
    pdata = pdev->dev.platform_data;
    if (pdata && (pdata->flags & IMXUART_HAVE_RTSCTS))
        sport->have_rtscts = 1;
    [...]
```

# Platform Driver

- On many embedded architectures, manual instantiation of platform devices was considered to be too verbose and not easily maintainable.
- Such architectures have moved, to use the *Device Tree*.
- It is a **tree of nodes** that models the hierarchy of devices in the system, from the devices inside the processor to the devices on the board.
- Each node can have a number of **properties** describing various properties of the devices: addresses, interrupts, clocks, etc.
- At boot time, the kernel is given a compiled version, the **Device Tree Blob**, which is parsed to instantiate all the devices described in the DT.
- On ARM, they are located in `arch/arm/boot/dts/`.

# Platform Driver

```
uart0: serial@44e09000 {
        compatible = "ti,omap3-uart";
        ti,hwmods = "uart1";
        clock-frequency = <48000000>;
        reg = <0x44e09000 0x2000>;
        interrupts = <72>;
        status = "disabled";
};
```

- `serial@44e09000` is the **node name**
- `uart0` is a **label**, that can be referred to in other parts of the DT as `&uart0`
- other lines are **properties**. Their values are usually strings, list of integers, or references to other nodes.

# Platform Driver

- Each particular hardware platform has its own *device tree*.
- However, several hardware platforms use the same processor, and often various processors in the same family share a number of similarities.
- To allow this, a *device tree* file can include another one. The trees described by the including file overlays the tree described by the included file. This can be done:
  - Either by using the `/include/` statement provided by the Device Tree language.
  - Either by using the `#include` statement, which requires calling the C preprocessor before parsing the Device Tree.

  Linux currently uses either one technique or the other (different from one ARM subarchitecture to another, for example).

# Platform Driver

```
/ {
    compatible = "ti,am33xx";
    [...]
    ocp {
        [...]
        uart0: serial@0 {
            compatible = "ti,am3352-uart", "ti,omap3-uart";
            reg = <0x0 0x1000>;
            interrupts = <72>;
            status = "disabled";
            [...]
        };
    };
};
```

am33xx-l4.dtsi (included by am33xx.dtsi)

Definition of the AM33xx SoC

+

```
[...]
&uart0 {
        pinctrl-names = "default";
        pinctrl-0 = <&uart0_pins>;
        status = "okay";
};
```

am335x-bone-common.dtsi

Common definitions for BeagleBone boards

+

```
#include "am33xx.dtsi"
#include "am335x-bone-common.dtsi"
#include "am335x-boneblack-common.dtsi"

/ {
        model = "TI AM335x BeagleBone Black";
        compatible = "ti,am335x-bone-black",
                     "ti,am335x-bone",
                     "ti,am33xx";
};

[...]
```

am335x-boneblack.dts

Definition for BeagleBone Black

=

Compiled DTB

```
/ {
    compatible = "ti,am335x-bone-black", "ti,am335x-bone", "ti,am33xx";
    model = "TI AM335x BeagleBone Black";
    [...]
    ocp {
        [...]
        uart0: serial@0 {
            compatible = "ti,am3352-uart", "ti,omap3-uart";
            reg = <0x0 0x1000>;
            interrupts = <72>;
            pinctrl-names = "default";
            pinctrl-0 = <&uart0_pins>;
            status = "okay";
        };
    };
};
```

am335x-boneblack.dtb

Note: the real DTB is in binary format. Here we show the text equivalent of the DTB contents;

# Platform Driver

- With the *device tree*, a *device* is bound to the corresponding *driver* using the **compatible** string.
- The `of_match_table` field of `struct device_driver` lists the compatible strings supported by the driver. `drivers/tty/serial/omap-serial.c` example:

```
#if defined(CONFIG_OF)
static const struct of_device_id omap_serial_of_match[] = {
        { .compatible = "ti,omap2-uart" },
        { .compatible = "ti,omap3-uart" },
        { .compatible = "ti,omap4-uart" },
        {},
};
MODULE_DEVICE_TABLE(of, omap_serial_of_match);
#endif
static struct platform_driver serial_omap_driver = {
        .probe          = serial_omap_probe,
        .remove         = serial_omap_remove,
        .driver         = {
                .name   = DRIVER_NAME,
                .pm     = &serial_omap_dev_pm_ops,
                .of_match_table = of_match_ptr(omap_serial_of_match),
        },
};
```

- Note: the `of_match_ptr()` macro instantiates to `NULL` when `CONFIG_OF` is not set.

# Platform Driver

- The drivers will use the same mechanism that we saw previously to retrieve basic information: interrupts numbers, physical addresses, etc.
- The available resources list will be built up by the kernel at boot time from the device tree, so that you don't need to make any unnecessary lookups to the DT when loading your driver.
- Any additional information will be specific to a driver or the class it belongs to, defining the *bindings*.

# Platform Driver

- **Describe hardware** (how the hardware is), not configuration (how I choose to use the hardware)
- **OS-agnostic**
  - For a given piece of HW, Device Tree should be the same for U-Boot, FreeBSD or Linux
  - There should be no need to change the Device Tree when updating the OS
- Describe **integration of hardware components**, not the internals of hardware components
  - The details of how a specific device/IP block is working is handled by code in device drivers
  - The Device Tree describes how the device/IP block is connected/integrated with the rest of the system: IRQ lines, DMA channels, clocks, reset lines, etc.
- Like all beautiful design principles, these principles are sometimes violated.

# Platform Driver

- How to write the correct nodes/properties to describe a given hardware platform ?
- **DeviceTree Specifications** → base Device Tree syntax + number of standard properties.
  - https://www.devicetree.org/specifications/
  - Not sufficient to describe the wide variety of hardware.
- **Device Tree Bindings** → documents that each specify how a piece of HW should be described
  - Documentation/devicetree/bindings/ in Linux kernel sources
  - Reviewed by DT bindings maintainer team
  - Legacy: human readable documents
  - New norm: YAML-written specifications



**Devicetree Specification**

*Release v0.3*

devicetree.org

13 February 2020

# Platform Driver RPI

```
/dts-v1/;
/plugin/;
/ {
    fragment@0 {
        target-path = "/";
        __overlay__ {
                hellokeys {
                        compatible = "arrow,hellokeys";
                };
        };
    };
};
```

**# dtc -@ -I dts -O dtb -o hello.dtbo hello.dts**

**# cp hello.dtbo /boot/overlays/**

**# dtoverlay hello.dtbo**

**# dtc -I fs /sys/firmware/devicetree/base > /tmp/test.txt**

**# vi /tmp/test.txt**

# Misc Driver

- The Miscellaneous Framework is an interface exported by the Linux kernel that allows modules to register their individual minor numbers.

- The device driver implemented as a miscellaneous character uses the major number allocated by the Linux kernel for miscellaneous devices. This eliminates the need to define an unique major number for the driver; this is important, as a conflict between major numbers has become increasingly likely, and use of the misc device class is an effective tactic.

- Each probed device is dynamically assigned a minor number, and is listed with a directory entry within the sysfs pseudo-filesystem under /sys/class/misc/.

- Major number 10 is officially assigned to the misc driver. Modules can register individual minor numbers with the misc driver and take care of a small device, needing only a single entry point.

# Misc Driver

- A misc device is defined by a struct miscdevice structure in include/linux/miscdevice.h:

```
struct miscdevice {
        int minor;
        const char *name;
        const struct file_operations *fops;
        struct list_head list;
        struct device *parent;
        struct device *this_device;
        const char *nodename;
        umode_t mode;
};
```

**Where:**

**minor:** is the minor number being registered

# Misc Driver

**name:** is the name for this device, found in the /proc/misc file

**fops:** is a pointer to the struct file_operations structure

**parent:** is a pointer to a struct device structure that represents the hardware device exposed by this driver

- The misc driver exports two functions, misc_register() and misc_deregister(), to register and unregister their own minor number. These functions are defined as function prototypes in **include/linux/miscdevice.h** and defined as a function in **drivers/char/misc.c**:

**int misc_register(struct miscdevice *misc);**

**int misc_deregister(struct miscdevice *misc);**

- The misc_register() function registers a miscellaneous device with the kernel. If the minor number is set to **MISC_DYNAMIC_MINOR** a minor number is dynamically assigned and placed in the minor field of the struct miscdevice structure. For other cases, the minor number requested is used.

# Misc Driver

- The structure passed is linked into the kernel and may not be destroyed until it has been unregistered.

- The typical code sequence for assigning a dynamic minor number is as follows:

```
static struct miscdevice my_dev;
int init_module(void)
{
    my_dev.minor = MISC_DYNAMIC_MINOR;
    my_dev.name = "my_device";
   my_dev.fops = &my_fops;
   misc_register(&my_dev);
   pr_info("my: got minor %i\n", my_dev.minor);
   return 0;
}
```

# Thank you